Université Abdelmalek Essaâdi
École Nationale des Sciences Appliquées – Al Hoceima
TP – J2EE, Java EE (JEE) ou Jakarta EE
TDIA2-S4 / 2025-26
Pr. Mohamed CHERRADI

TP N° 2:

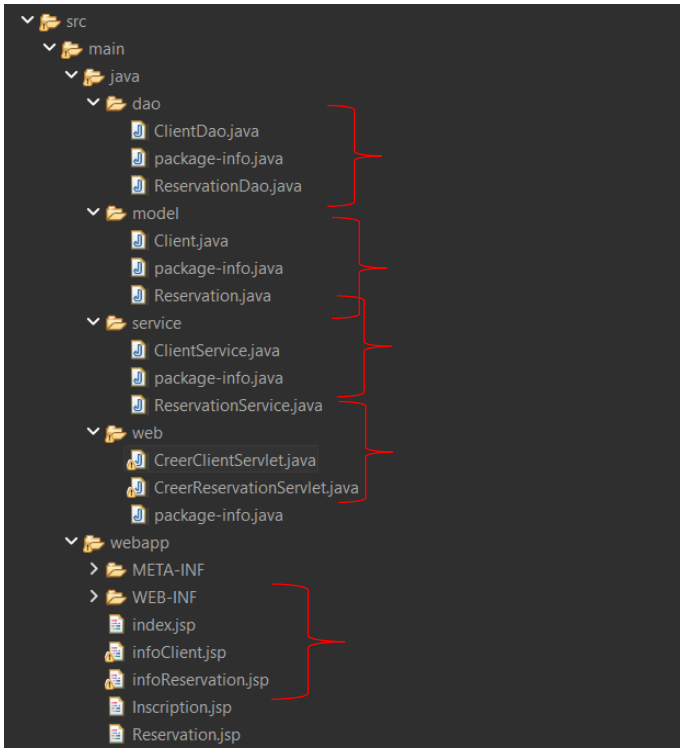
Réalisés par: Asmae Daouad.

1. Objectif du TP:

L'objectif de ce TP est de se familiariser avec les technologies Servlet et JSP pour le développement d'une application web dynamique en Java. L'application permet d'ajouter des clients et des réservations via des formulaires, et d'afficher les données saisies.

2. Architecture du projet:

L'application suit une architecture basée sur les composants suivants :



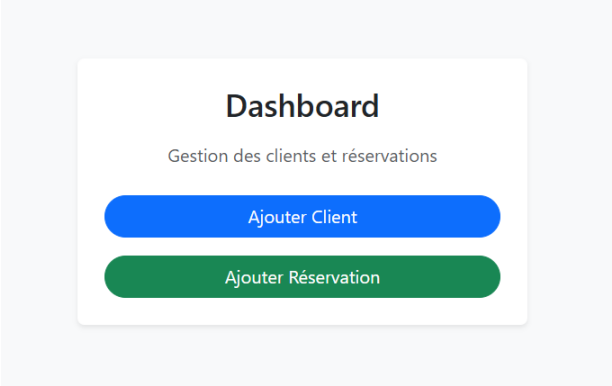
- dao : gère le stockage des données en mémoire via des listes (ArrayList) maintenues en session
- model : contient les classes métier Client et Reservation
- service : contient la logique métier et fait le lien entre les servlets et le dao
- web : contient les servlets qui traitent les requêtes HTTP
- WebContent : contient les pages JSP pour l'affichage (vue)

3. Réalisation:

3.1 Page d'accueil (index.jsp):

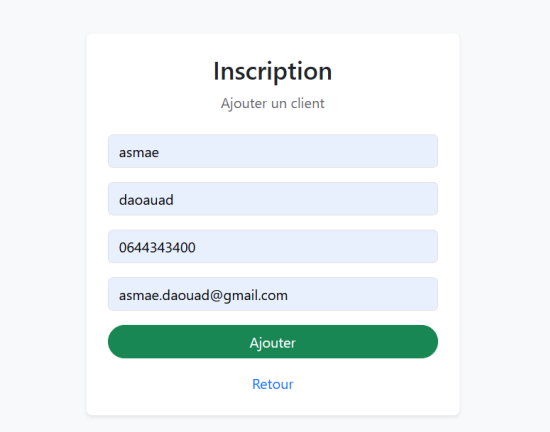
La page d'accueil affiche deux boutons permettant à l'utilisateur de naviguer vers le formulaire d'ajout d'un client ou vers le formulaire d'ajout d'une reservation.

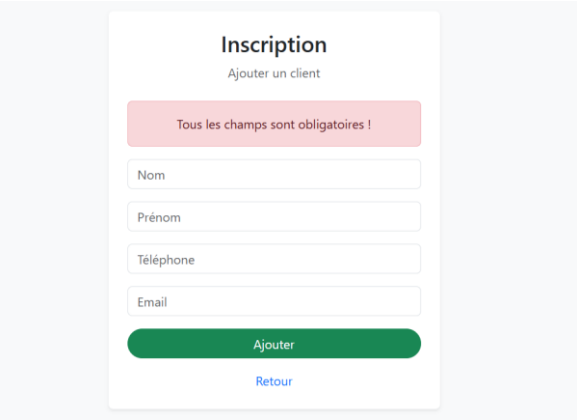
```html
<body class="d-flex justify-content-center align-items-center vh-100 bg-light">

<div class="card shadow-sm border-0 text-center" style="width: 400px;">
    <div class="card-body p-4">

        <h3 class="mb-3">Dashboard</h3>
        <p class="text-muted mb-4">Gestion des clients et réservations</p>

        <div class="d-grid gap-3">
            <a href="Inscription.jsp" class="btn btn-primary rounded-pill">
                Ajouter Client
            </a>

            <a href="Reservation.jsp" class="btn btn-success rounded-pill">
                Ajouter Réservation
            </a>
        </div>

    </div>
</div>

</body>
```



3.2 Formulaire Client (Inscription.jsp):

Cette page JSP affiche un formulaire HTML avec les champs : Nom, Prénom, Téléphone et E-mail.



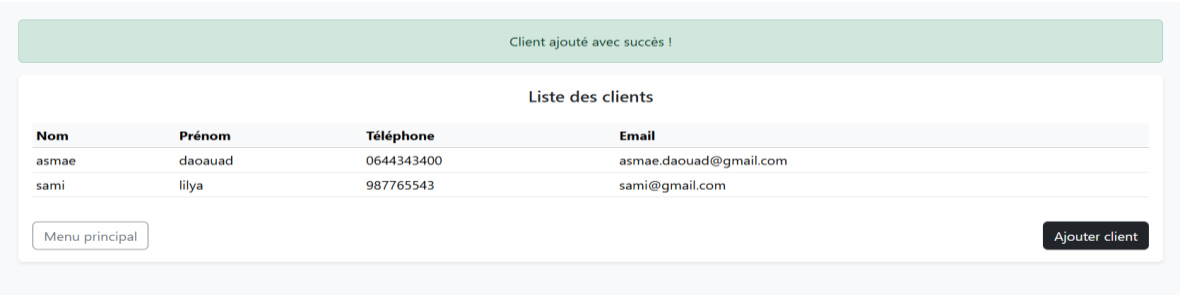


En cas d'erreur de validation (champ vide), un message d'alerte Bootstrap est affiché en rouge. Le formulaire soumet les données en POST vers la servlet CreerClientServlet.

```jsp
<% if(request.getAttribute("error") != null) { %>
    <div class="alert alert-danger text-center">
        <%= request.getAttribute("error") %>
    </div>
<% } %>
```

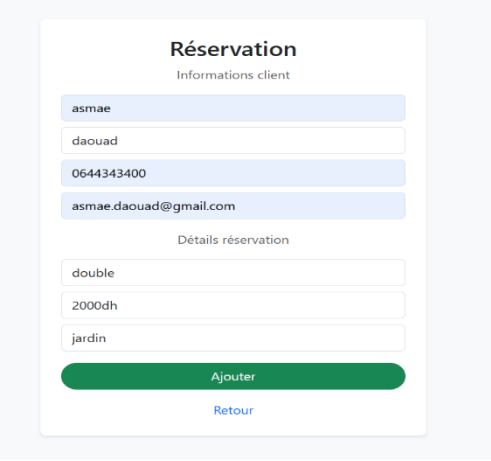
3.3 Affichage Client (infoClient.jsp):

Cette page reçoit les attributs transmis par la servlet et les affiche dans un tableau .



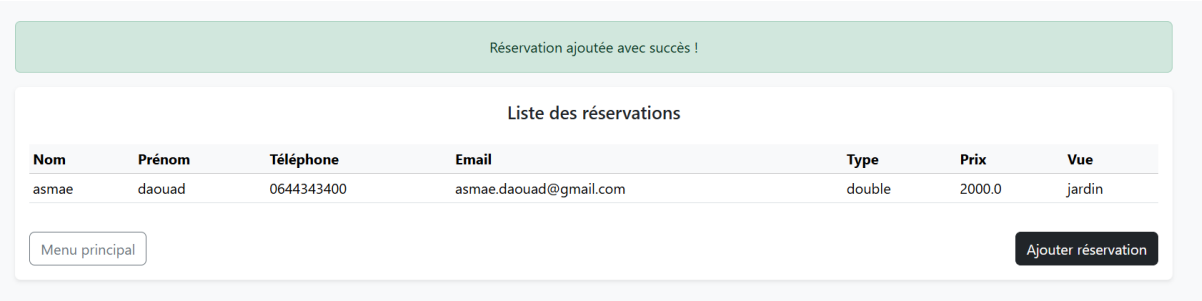
3.4 Formulaire Réservation (Reservation.jsp):

Cette page JSP affiche un formulaire avec les champs du client (nom, prénom, téléphone, e-mail) plus les champs spécifiques à la réservation



3.5 Affichage Réservation (infoReservation.jsp):

Cette page affiche toutes les données de la réservation dans un tableau : nom, prénom, téléphone, e-mail, type, prix et vue.



4. Flux de fonctionnement

Le fonctionnement de l'application suit le cycle requête/réponse HTTP suivant :

- ✓ L'utilisateur accède à index.jsp et choisit une action (client ou réservation)
- ✓ Il remplit le formulaire JSP (Inscription.jsp ou Reservation.jsp) et soumet
- ✓ La servlet correspondante (CreerClientServlet ou CreerReservationServlet) reçoit la requête POST

```java
@WebServlet("/CreerClientServlet")
public class CreerClientServlet extends HttpServlet {

    private ClientService service = new ClientService();

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {


        String nom = request.getParameter("nom");
        String prenom = request.getParameter("prenom");
        String telephone = request.getParameter("telephone");
        String email = request.getParameter("email");


        String error = service.ajouterClient(nom, prenom, telephone, email);

        if (error != null) {
            request.setAttribute("error", error);
            request.getRequestDispatcher("Inscription.jsp").forward(request, response);
        } else {

            HttpSession session = request.getSession();
            session.setAttribute("clients", service.getAllClients());
            request.setAttribute("success", "Client ajouté avec succès !");
            request.setAttribute("clients", service.getAllClients());
            request.getRequestDispatcher("infoClient.jsp").forward(request, response);
        }
    }

}
```

```java
@WebServlet("/CreerReservationServlet")
public class CreerReservationServlet extends HttpServlet {

    private ReservationService service = new ReservationService();

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {


        String nom = request.getParameter("nom");
        String prenom = request.getParameter("prenom");
        String telephone = request.getParameter("telephone");
        String email = request.getParameter("email");


        String type = request.getParameter("type");
        String prix = request.getParameter("prix");
        String vue = request.getParameter("vue");


        String error = service.ajouterReservation(
                nom, prenom, telephone, email,
                type, prix, vue
        );

        if (error != null) {

            request.setAttribute("error", error);
            request.getRequestDispatcher("Reservation.jsp").forward(request, response);
        } else {

            HttpSession session = request.getSession();
            session.setAttribute("reservations", service.getAllReservations());
            request.setAttribute("success", "Réservation ajoutée avec succès !");
            request.setAttribute("reservations", service.getAllReservations());
            request.getRequestDispatcher("infoReservation.jsp").forward(request, response);
        }
```

- ✓ La servlet appelle le Service, qui appelle le DAO pour stocker l'objet dans la liste en session

```java
public class ClientDao {

    private static List<Client> clients = new ArrayList<>();

    public void addClient(Client client) {
        clients.add(client);
    }


    public List<Client> getAllClients() {
        return clients;
    }


    public Client getClientByName(String nom) {
        for (Client c : clients) {
            if (c.getNom().equalsIgnoreCase(nom)) {
                return c;
            }
        }
        return null;
    }


    public Client getClientByEmail(String email) {
        for (Client c : clients) {
            if (c.getEmail().equalsIgnoreCase(email)) {
                return c;
            }
        }
        return null;
    }
```

```java
public class ClientService {

    private ClientDao clientDao = new ClientDao();

    // Ajouter un client avec validation
    public String ajouterClient(String nom, String prenom, String telephone, String email) {


        if(nom == null || nom.isEmpty() ||
           prenom == null || prenom.isEmpty() ||
           telephone == null || telephone.isEmpty() ||
           email == null || email.isEmpty()) {

            return "Tous les champs sont obligatoires !";
        }

        Client client = new Client(nom, prenom, telephone, email);

        clientDao.addClient(client);

        return null;
    }

    // Récupérer tous les clients
    public List<Client> getAllClients() {
        return clientDao.getAllClients();
    }
```

```java
public class ReservationDao {

    // Stockage en mémoire
    private static List<Reservation> reservations = new ArrayList<>();

    // Ajouter une réservation
    public void addReservation(Reservation reservation) {
        reservations.add(reservation);
    }

    // Récupérer les réservations
    public List<Reservation> getAllReservations() {
        return reservations;
    }
}
```

```java
public class ReservationService {

    private ReservationDao reservationDao = new ReservationDao();
    private ClientDao clientDao = new ClientDao();  // AJOUTÉ

    // Ajouter une réservation avec validation
    public String ajouterReservation(String nom, String prenom, String telephone, String email,
                                     String type, String prixStr, String vue) {


        if(nom == null || nom.isEmpty() ||
           prenom == null || prenom.isEmpty() ||
           telephone == null || telephone.isEmpty() ||
           email == null || email.isEmpty() ||
           type == null || type.isEmpty() ||
           prixStr == null || prixStr.isEmpty() ||
           vue == null || vue.isEmpty()) {

            return "Tous les champs sont obligatoires !";
        }

        double prix;

        try {
            prix = Double.parseDouble(prixStr);
        } catch (NumberFormatException e) {
            return "Le prix doit être un nombre valide !";
        }


        Client client = clientDao.getClientByName(nom);

        // Si client n'existe pas, on en crée un nouveau
        if (client == null) {
            client = new Client(nom, prenom, telephone, email);
            clientDao.addClient(client);
        }

        Reservation reservation = new Reservation(client, type, prix, vue);

        reservationDao.addReservation(reservation);
```

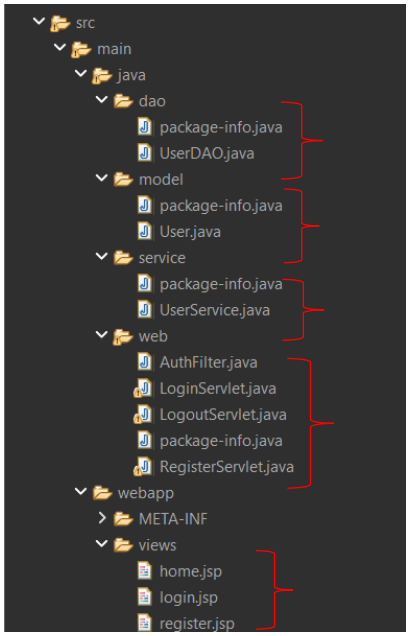
- ✓ 5. La servlet transmet la liste mise à jour via request.setAttribute() et redirige vers la JSP d'affichage
- ✓ 6. La JSP (infoClient.jsp ou infoReservation.jsp) affiche le message de succès et la liste complete

TP N° 3:

3.Objectif du TP :

L'objectif de ce TP est de réaliser une application web dynamique en Java permettant la gestion de l'authentification des utilisateurs. L'application propose un système complet d'inscription, de connexion et de déconnexion, avec une protection des pages via un filtre de session. Elle met en pratique les Servlets, les JSP, le pattern DAO/Service, et la gestion de session HTTP.

2. Architecture du projet:



→ Accès aux données (HashMap en mémoire)

→ Classe modèle utilisateur

→ Logique métier (inscription, connexion)
→ Filtre de protection des pages
→ Traitement de la connexion
→ Déconnexion et invalidation de session
→ Traitement de l'inscription

→ Page d'accueil protégée
→ Formulaire de connexion
→ Formulaire d'inscription

3. Réalisation:

3.1 Modèle – User.java:

La classe User représente un utilisateur de l'application. Elle contient quatre attributs : username, password, email et phone, avec un constructeur, des getters et des setters.

```java
package model;

public class User {

    private String username;
    private String password;
    private String email;
    private String phone;

    // Constructeur
    public User(String username, String password, String email, String phone) {
        this.username = username;
        this.password = password;
        this.email = email;
        this.phone = phone;
    }
```

3.2 Couche DAO – UserDAO.java

Le DAO (Data Access Object) gère le stockage des utilisateurs en mémoire via une HashMap statique. Il propose trois méthodes : addUser() pour ajouter un utilisateur, getUserByUsername() pour le récupérer par son identifiant, et exists() pour vérifiers'il existe déjà.

```java
package dao;

import java.util.Map;

public class UserDAO{

    private static Map<String,User> users= new HashMap<>();

    // Ajouter User

    public void addUser(User user) {
        users.put(user.getUsername(), user);
    }


     // récupérer un utilisateur par username


    public User getUserByUsername(String username) {
        return users.get(username);

    }

     // vérifier si utilisateur existe

    public boolean exists(String username) {

        return users.containsKey(username);

    }
```

3.3 Couche Service – UserService.java:

La couche service contient la logique métier de l'application. La méthode register() vérifie si le username existe déjà avant d'ajouter l'utilisateur. La méthode login() compare le mot de passe saisi avec celui stocké pour authentifier l'utilisateur.

```java
package service;

import dao.UserDAO;

public class UserService {

    private UserDAO userDao;

    // constructeur
    public UserService() {
        this.userDao = new UserDAO();
    }

    // Inscription
    public boolean register(User user) {

        if(userDao.exists(user.getUsername())) {
            return false;
        }

        userDao.addUser(user);
        return true;
    }

    // Connexion
    public boolean login(String username, String password) {
        User user = userDao.getUserByUsername(username);
        if(user != null && user.getPassword().equals(password)) {
            return true;
        }
        return false;
    }

    // Récupérer l'utilisateur complet
    public User getUser(String username) {
        return userDao.getUserByUsername(username);
    }
}
```

3.4 Filtre de sécurité – AuthFilter.java:

Le filtre AuthFilter intercepte toutes les requêtes grâce à l'annotation @WebFilter("/*"). Il laisse passer les requêtes vers login.jsp, register.jsp et les servlets Login/Register. Pour toutes les autres pages, il vérifie la presence de l'attribut "user" dans la session. Si l'utilisateur n'est pas connecté, il est redirigé vers login.jsp.

```java
@WebFilter("/*")
public class AuthFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse res = (HttpServletResponse) response;

        String path = req.getRequestURI();
        String contextPath = req.getContextPath();
        String relativePath = path.substring(contextPath.length());


        if(relativePath.endsWith("login.jsp") ||
           relativePath.endsWith("register.jsp") ||
           relativePath.contains("/Login") ||
           relativePath.contains("/Register")) {
            chain.doFilter(request, response);
            return;
        }

        HttpSession session = req.getSession(false);
        if(session != null && session.getAttribute("user") != null) {
            chain.doFilter(request, response);
        } else {
            res.sendRedirect(contextPath + "/views/login.jsp");
        }
    }
}
```

3.5 Inscription – RegisterServlet.java:

La servlet RegisterServlet traite le formulaire d'inscription (POST). Elle récupère les paramètres username, password, email et phone, vérifie si le username existe déjà, crée un objet User et appelle userService.register(), elle redirige vers login.jsp. Si le username est déjà pris, un message d'erreur est affiché.

```java
@WebServlet("/Register")
public class RegisterServlet extends HttpServlet {

    private UserService userService;

    @Override
    public void init() throws ServletException {
        userService = new UserService();
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.sendRedirect(request.getContextPath() + "/views/register.jsp");
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");
        String email    = request.getParameter("email");
        String phone    = request.getParameter("phone");

        if(userService.getUser(username) != null) {
            request.setAttribute("error", "Username already exists");
            request.getRequestDispatcher("/views/register.jsp").forward(request, response);
            return;
        }

        User newUser = new User(username, password, email, phone);
        userService.register(newUser);
        response.sendRedirect(request.getContextPath() + "/views/login.jsp");
```

3. 6 Connexion – LoginServlet.java:

La servlet LoginServlet traite la connexion (POST). Elle appelle userService.login() pour vérifier les identifiants. Si la connexion réussit, elle crée une session HTTP et stocke l'objet User connecté avec session.setAttribute("user", user), puis redirige vers home.jsp. Sinon, un message d'erreur est affiché.

3. 7 Déconnexion – LogoutServlet.java:

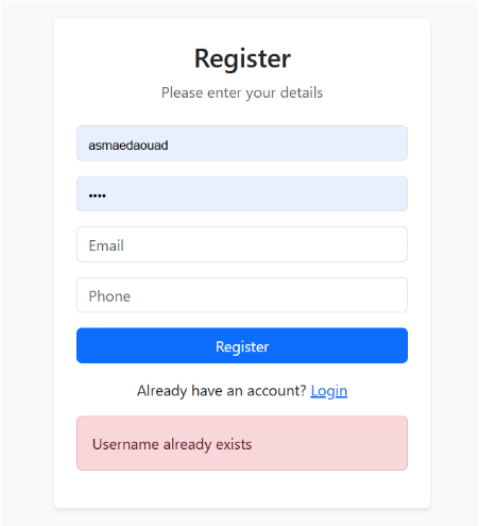
La servlet LogoutServlet invalide la session HTTP courante via session.invalidate(), ce qui supprime toutes les données de session incluant l'utilisateur connecté. L'utilisateur est ensuite redirigé vers login.jsp.

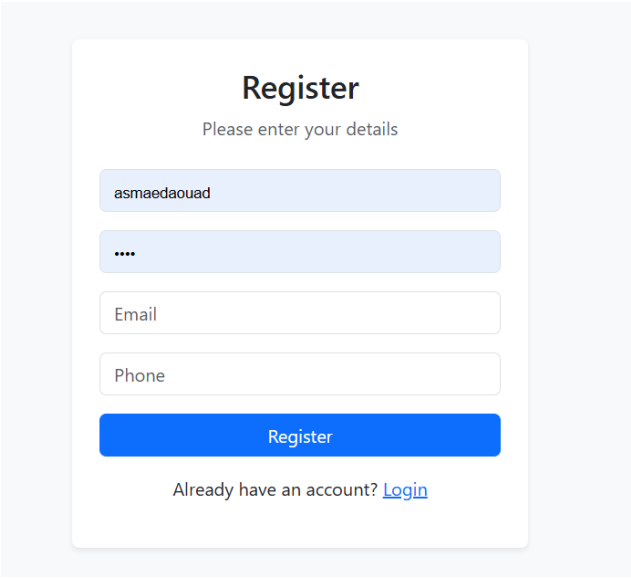
```java
@WebServlet("/Logout")
public class LogoutServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session != null) {
            session.invalidate();
        }
        response.sendRedirect(request.getContextPath() + "/views/login.jsp");
    }
}
```

3. 8 Vues JSP:

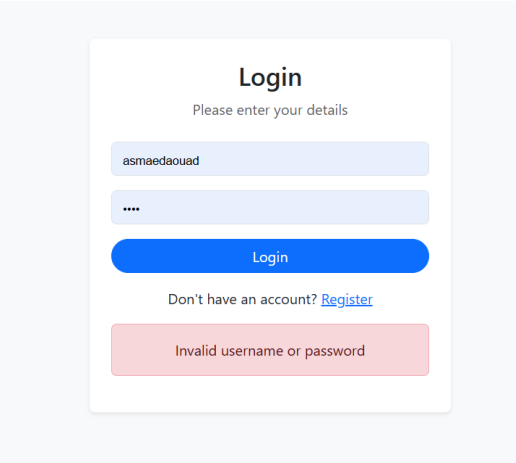
L'application contient trois vues JSP situées dans le dossier *webapp/views/* :

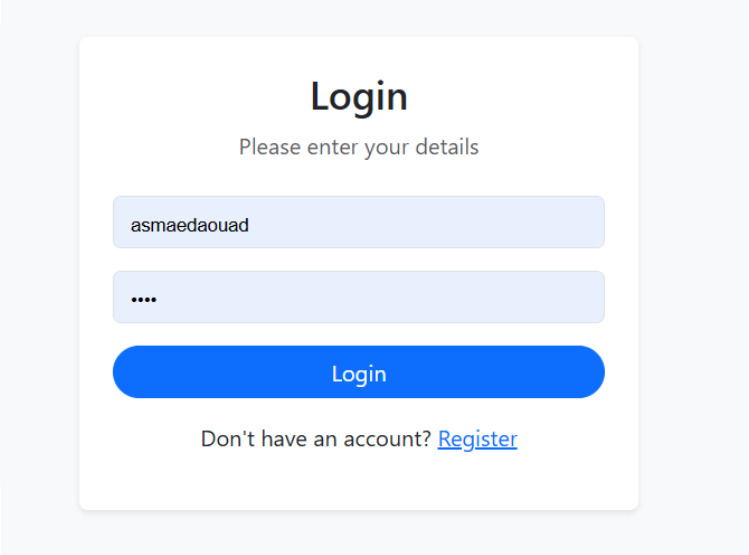
- register.jsp : formulaire d'inscription avec quatre champs (username, password, email, phone).
- Affiche un message d'erreur Bootstrap si le username est déjà pris.



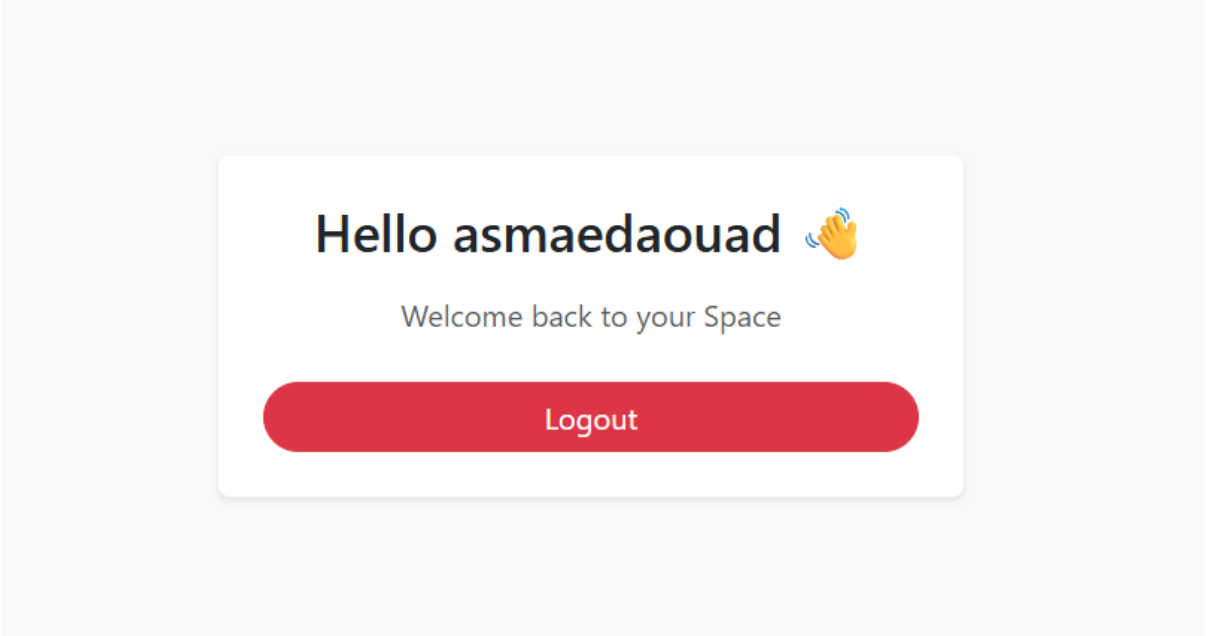


- login.jsp : formulaire de connexion avec deux champs (username, password). Affiche un message d'erreur si les identifiants sont incorrects. Contient un lien vers la page d'inscription.





- home.jsp : page protégée accessible uniquement après connexion. Affiche le username de l'utilisateur connecté récupéré depuis la session, et un bouton de déconnexion (Logout) qui soumet un formulaire POST vers la servlet /Logout.



Conclusion:

Ces TP m'a permis de mettre en pratique une architecture web Java complète en couches (Model – DAO – Service – Web). J'ai appris à gérer l'authentification des utilisateurs avec les sessions HTTP, à protéger les pages sensibles grâce à un filtre Servlet, et à organiser le code de manière structurée. L'utilisation de Bootstrap a permis d'obtenir une interface propre et responsive. Ce projet constitue une base solide pour développer des applications web Java plus avancées avec une base de données réelle.